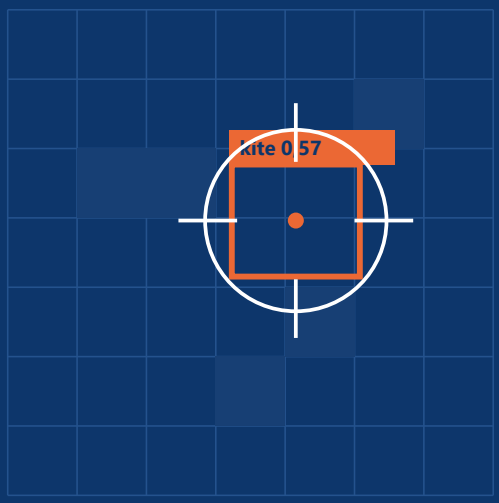
PÓS-GRADUAÇÃO · ENGENHARIA APLICADA A IA · UNIPDS

Vencendo o Duck Hunt com Visão Computacional

Guia de estudo do exemplo 02 (parte 1): um detector de objetos YOLOv5 pré-treinado, rodando com TensorFlow.js num Web Worker, olha a tela do jogo, encontra os patos e atira sozinho.



Material de estudo · outubro de 2026

O PROJETO EM UMA FRASE

Nenhum modelo foi treinado: um detector que **nunca viu um pato de 8 bits** é usado como está, e o truque de engenharia é descobrir que ele chama o pato de **"pipa" (kite)**, filtrar essa classe e transformar a caixa detectada num **clique**.

O PROJETO EM NÚMEROS

640^2	80	1,88 M	200 ms	0,4	60 px
entrada do modelo (pixels)	classes do COCO, só 1 usada	parâmetros no YOLOv5n	intervalo entre capturas	limiar de confiança	raio do tiro (nível 1)

Os números do experimento (seção 8) vêm de uma execução real do **mesmo modelo e do mesmo pré-processamento** do `worker.js` (TensorFlow.js 4.22.0), em 40 cenas montadas com os sprites originais do jogo (80 patos). Cenas geradas pelo mesmo script dão os mesmos números; frames do jogo ao vivo variam, mas as conclusões se mantêm.

Sumário

01 O problema: jogar com os olhos . . . 3
02 O pipeline de ponta a ponta . . . 4
03 Capturando a tela e conversando com o Worker . . . 5
04 Pré-processamento: da imagem ao tensor . . . 6
05 O modelo: YOLOv5n . . . 7
06 Pós-processamento: de 100 caixas a um ponto . . . 8
07 Da previsão ao tiro . . . 9
08 Experimento: o que o modelo realmente vê . . . 10
09 Avaliação crítica: o que um engenheiro de IA perguntaria . . . 12
10 Engenharia: IA rodando no navegador . . . 13
11 Exercícios práticos . . . 14
12 Glossário . . . 15
13 Perguntas de revisão . . . 16
14 Resumo para revisar em 5 minutos . . . 18

COMO ESTUDAR COM ESTE GUIA

1. Leia uma seção por vez com o arquivo citado nos quadros “No código” aberto ao lado. Teoria e linha de código juntas é o que fixa.

2. Rode o jogo (`npm start`) com o console aberto: cada `AI predicted at` é uma detecção que virou tiro.

3. Responda às perguntas da seção 13 sem olhar e só depois confira o gabarito.

4. Faça pelo menos dois exercícios da seção 11. Mudar o limiar ou a classe e ver a mira reagir é o jeito mais rápido de entender precisão e recall.

SEÇÃO 01

O problema: jogar com os olhos

Para vencer um jogo, um agente precisa repetir três passos: **perceber** o estado do jogo, **decidir** o que fazer e **agir**. Neste projeto a percepção é feita só com a imagem da tela, como um humano faria. É isso que torna a ideia reaproveitável em "qualquer jogo".

1.1 Três formas de fazer uma IA jogar

Abordagem	Como funciona	Precisa de	Quando usar
Ler o estado interno	Lê a posição dos patos direto das variáveis do jogo (`stage.ducks`).	Acesso ao código	Bots e testes; não generaliza para outro jogo
Visão computacional (este projeto)	Um detector olha o frame, encontra os alvos e uma regra fixa decide o clique.	Um modelo de visão; regras de ação	Jogos em que "achar o alvo" resolve quase tudo
Aprendizado por reforço	O agente testa ações, recebe recompensa (pontos) e aprende uma política.	Milhares de partidas simuladas	Jogos com estratégia e consequências de longo prazo

1.2 Classificar, detectar ou segmentar?

Cada tarefa de visão responde a uma pergunta diferente. Para atirar, não basta saber *que* existe um pato na tela: é preciso saber *onde*. Por isso o projeto usa **detecção de objetos**, que devolve uma caixa (bounding box), uma classe e uma confiança para cada objeto encontrado.

Tarefa	Pergunta	Saída	Serve para atirar?
Classificação	O que há nesta imagem?	Uma classe + probabilidade	Não: não diz onde
Detecção	**O que há e onde está?**	**Caixas + classes + confiança**	**Sim: o centro da caixa vira o clique**
Segmentação	Quais pixels são de qual objeto?	Máscara por pixel	Sim, mas é mais cara do que o necessário

A IDEIA CENTRAL

O projeto **não treina nada**. Ele usa **transferência zero-shot**: um modelo treinado em fotos reais (COCO) é aplicado a um domínio que nunca viu (pixel art de 1984). O trabalho de engenharia está em volta do modelo: capturar a imagem, prepará-la, interpretar a saída e transformar a detecção numa ação.

SEÇÃO 02

O pipeline de ponta a ponta

Do pixel ao tiro são oito etapas. As quatro de cima rodam numa thread separada (o Web Worker) para que o jogo continue liso a 60 quadros por segundo; as de baixo voltam para a thread principal, onde vive o jogo.

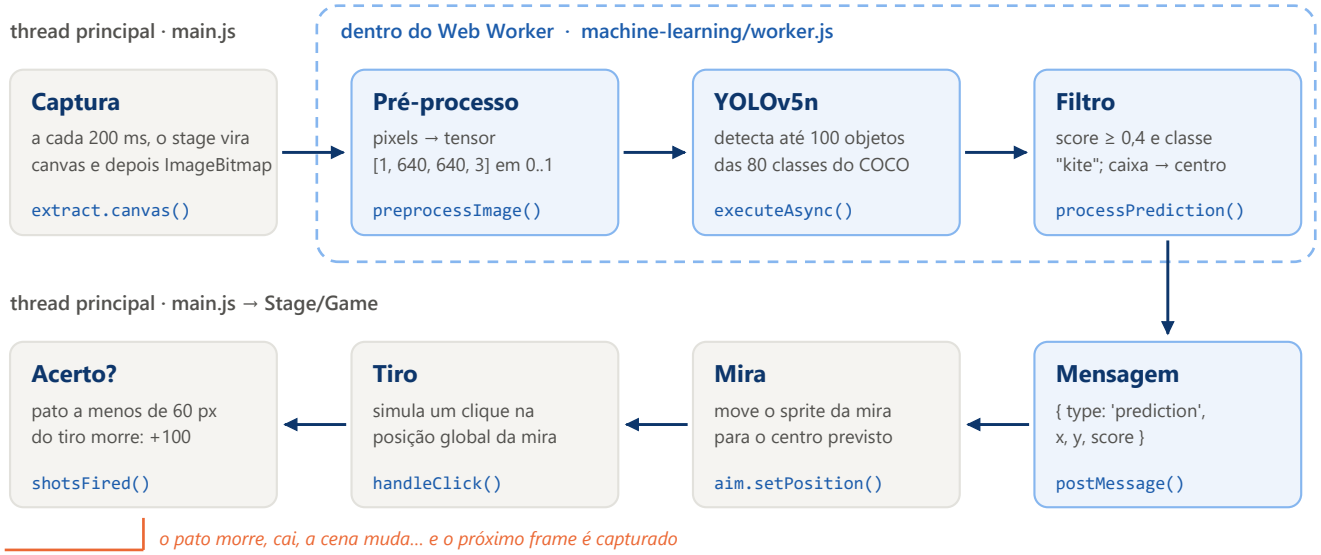


Figura 1. O ciclo completo de percepção → decisão → ação. A seta laranja fecha o laço: o tiro muda a cena, que é capturada de novo.

Arquivo	Papel
`main.js` (raiz)	Cria o `Game`, espera os sprites carregarem e chama a IA: `await main(game)`.
`machine-learning/main.js`	Loop de captura (`setInterval`), cria o worker, recebe as previsões e dispara os tiros.
`machine-learning/worker.js`	Carrega o TF.js e o YOLOv5n, pré-processa a imagem, roda a inferência e filtra o resultado.
`machine-learning/layout.js`	HUD da IA no canto superior direito: score e coordenadas da última previsão.
`machine-learning/yolov5n_web_model/`	`model.json` (grafo), 2 arquivos `.bin` (7,5 MB de pesos) e `labels.json` (80 classes).
`src/modules/Game.js · Stage.js`	O jogo original: `handleClick`, `shotsFired`, balas, pontuação e níveis.
`webpack.config.js`	`CopyWebpackPlugin` copia a pasta do modelo para `dist/`, onde o worker a busca com `fetch`.

SEÇÃO 03

Capturando a tela e conversando com o Worker

O JavaScript da página roda numa única thread: se a inferência rodasse nela, o jogo congelaria a cada previsão. O Web Worker é uma segunda thread; a página e ele só conversam por mensagens.

machine-learning/main.js

```
const worker = new Worker(new URL('./worker.js', import.meta.url), { type: 'module' });

setInterval(async () => {
    const canvas = game.app.renderer.extract.canvas(game.stage);  // desenha o stage num canvas
    const bitmap = await createImageBitmap(canvas);                // imagem pronta para a GPU
    worker.postMessage({ type: 'predict', image: bitmap }, [bitmap]); // transfere, não copia
}, 200); // every 200ms
```

3.1 Três detalhes que fazem diferença

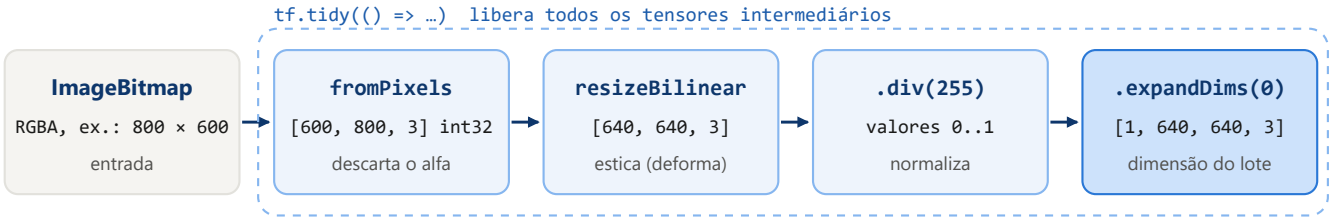
- `extract.canvas(game.stage)` renderiza só o *container* do jogo. A cor de fundo azul é do *renderer*, não um sprite do stage, por isso o céu sai **transparente**. Ao virar tensor, o canal alfa é descartado e o transparente vira **preto** (Figura 4, seção 8).
- **Objeto transferível.** O segundo argumento `[bitmap]` move o `ImageBitmap` para o worker sem copiar os bytes. Depois disso a página não pode mais usá-lo. Com um frame de 800 × 600 × 4 bytes ≈ 1,9 MB, 5 vezes por segundo, copiar seria desperdício.
- **Protocolo por `type`.** A página envia `{ type: 'predict' }`; o worker responde `{ type: 'model-loaded' }` uma vez e `{ type: 'prediction', x, y, score }` para cada pato encontrado. Mensagens de outros tipos são ignoradas.

> **ATENÇÃO**
> `setInterval` dispara a cada 200 ms **sem esperar** o worker terminar. Se uma inferência demorar mais que isso, as mensagens se acumulam na fila do worker e ele passa a responder sobre frames velhos. A seção 9 mostra o efeito e como resolver com *backpressure*.

SEÇÃO 04

Pré-processamento: da imagem ao tensor

Uma rede neural só recebe números num formato exato. O YOLOv5n exportado espera um tensor **[1, 640, 640, 3]** de floats entre 0 e 1. A função `preprocessImage` faz essa conversão em quatro operações.



[lote, altura, largura, canais] · NHWC: a convenção do TensorFlow para imagens

800 × 600 (4:3) vira 640 × 640 (1:1): tudo fica 20% mais estreito e 7% mais alto. O YOLO original usaria letterbox (borda).

Figura 2. As quatro transformações do pré-processamento e o formato do tensor depois de cada uma.

machine-learning/worker.js

```
function preprocessImage(input) {
    return tf.tidy(() => {
        const image = tf.browser.fromPixels(input)       // [H, W, 3], inteiros 0..255
        return tf.image
            .resizeBilinear(image, [INPUT_MODEL_DIMENTIONS, INPUT_MODEL_DIMENTIONS]) // [640, 640, 3]
            .div(255)                                    // 0..1, como no treino do YOLO
            .expandDims(0)                               // [1, 640, 640, 3]: lote de 1 imagem
    })
}
```

4.1 Por que cada passo existe

Passo	Motivo
`fromPixels`	Converte a imagem num tensor. Mantém só RGB: o alfa some, e é aí que o céu transparente vira preto.
`resizeBilinear`	O modelo foi exportado com entrada fixa de 640 × 640. Bilinear calcula cada pixel novo pela média ponderada dos 4 vizinhos.
`.div(255)`	O YOLO foi treinado com pixels em 0..1. Entrar com 0..255 seria entregar dados numa escala que a rede nunca viu.
`.expandDims(0)`	Redes processam **lotes**. Mesmo com uma imagem só, o tensor precisa da dimensão do lote na frente.
`tf.tidy`	Os tensores ficam na memória da GPU (backend WebGL) e o coletor de lixo do JS não os libera. `tidy` descarta todos os intermediários e mantém só o retorno.

PARA A PROVA: RESIZE × LETTERBOX

Ao esticar 800 × 600 para 640 × 640, a imagem **deforma** (aspect ratio de 4:3 para 1:1). O YOLO original usa **letterbox**: redimensiona mantendo a proporção e preenche o resto com borda cinza. Deformar funciona aqui porque as coordenadas de saída são normalizadas (0..1) e são multiplicadas pela largura e altura originais, o que desfaz a deformação. Mas a rede vê patos “achatados”, o que pode reduzir a confiança.

SEÇÃO 05

O modelo: YOLOv5n

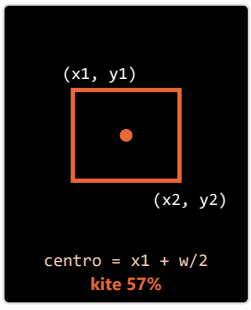
YOLO (*You Only Look Once*) é uma família de detectores **de um estágio**: uma única passada da rede pela imagem produz todas as caixas. O "n" de *nano* é a menor versão, pensada para rodar em celular e navegador.

5.1 Como um detector de um estágio pensa

1. **Backbone** (convoluções): extrai características da imagem em várias escalas, de bordas e cores até formas inteiras.
2. **Neck**: combina mapas de escalas diferentes para achar objetos pequenos e grandes.
3. **Head**: a imagem é dividida em grades (80 × 80, 40 × 40 e 20 × 20 para 640 px). Cada célula prevê caixas a partir de *anchors* (formatos pré-definidos), com uma confiança de "há objeto" e uma probabilidade para cada uma das 80 classes.
4. **NMS (Non-Maximum Suppression)**: várias células detectam o mesmo pato. O NMS mantém a caixa de maior confiança e apaga as que se sobrepõem a ela.

O que olhar no `model.json`	Valor neste projeto	O que significa
Formato	`graph-model` (TF.js Converter v4.5)	Grafo congelado só para inferência; não dá para treinar com `fit`.
Parâmetros	1.884.494 em 270 tensores	≈ 7,5 MB em float32 (os dois `.bin`). Uma rede de recomendação do exemplo 01 tinha 14 mil.
Convoluções	60 `_FusedConv2D`	Convolução + bias + ativação fundidas numa operação só, mais rápida.
Ativação	66 `Sigmoid` + 71 `Mul`	É a SiLU: x · sigmoid(x), a ativação usada pelo YOLOv5.
Laços	`TensorList*, Merge, Switch`	Fluxo de controle do NMS embutido no grafo; por isso é obrigatório usar `executeAsync`.
Saídas	4 tensores	Já pós-NMS: no máximo 100 caixas, scores abaixo de ≈ 0,25 já cortados.

boxes	`[1, 100, 4]`	x1, y1, x2, y2 normalizados (0..1)
scores	`[1, 100]`	confiança de cada caixa
classes	`[1, 100]`	índice da classe (0..79)
valid_detections	`[1]`	quantas das 100 linhas valem



O worker lê só as 3 primeiras saídas (output.slice(0, 3)); a 4ª é descartada e o laço percorre as 100 linhas.

Figura 3. As 4 saídas do modelo. As caixas vêm normalizadas: (0, 0) é o canto superior esquerdo e (1, 1) o inferior direito.

5.2 Carregar, aquecer, inferir

machine-learning/worker.js · loadModelAndLabels()

```
_labels = await (await fetch(LABELS_PATH)).json()
_model = await tf.loadGraphModel(MODEL_PATH)       // baixa model.json + shards .bin

const dummyInput = tf.ones(_model.inputs[0].shape)  // [1, 640, 640, 3]
await _model.executeAsync(dummyInput)               // warmup
tf.dispose(dummyInput)
```

CONCEITO-CHAVE

Por que o warmup? No backend WebGL a primeira execução compila os *shaders* de cada operação e aloca texturas na GPU. Isso pode levar segundos. Rodar uma imagem falsa no carregamento faz esse custo acontecer antes do primeiro pato aparecer.

Por que `executeAsync` e não `predict`? `predict`/`execute` são síncronos e não suportam operações de fluxo de controle. Como o NMS está dentro do grafo, o TF.js exige a versão assíncrona.

SEÇÃO 06

Pós-processamento: de 100 caixas a um ponto

A saída crua são números. O worker transforma em decisões com três filtros e uma conta de geometria.

machine-learning/worker.js

```
function * processPrediction({boxes, scores, classes,}, width, height){
    for (let index = 0; index < scores.length; index++){
        if(scores[index] < CLASS_THRESHOLD) continue    // 1. confiança mínima: 0,4
        const label = _labels[classes[index]]
        if(label != 'kite') continue                     // 2. só a classe "pipa"
        let [x1, y1, x2, y2] = boxes.slice(index * 4, (index +1) * 4)
        x1 *= width;  x2 *= width;  y1 *= height;  y2 *= height   // 3. 0..1 → pixels
        const centerX = x1 + (x2 - x1) / 2
        const centerY = y1 + (y2 - y1) / 2
        yield { x: centerX, y: centerY, score: (scores[index] * 100).toFixed(2) }
    }
}
```

6.1 Por que “kite”?

O COCO não tem a classe “pato”. As mais próximas seriam `bird` (índice 14) ou `kite` (pipa, índice 33). Na prática, um pato pixelado contra um fundo liso parece muito mais uma pipa de foto (forma pequena e colorida contra o céu) do que um pássaro de foto. Essa escolha foi descoberta **empiricamente**: olhando o que o modelo dizia e filtrando a classe que mais aparecia em cima dos patos. A seção 8 mede isso.

6.2 Por que um generator (`function*`)?

Cada `yield` entrega uma previsão assim que ela é calculada, sem montar um array intermediário. O `for…of` do `onmessage` consome uma por vez e faz um `postMessage` para cada pato. Resultado: **um frame com dois patos gera duas mensagens, e cada mensagem vira um tiro**.

ATENÇÃO

O modelo devolve `valid_detections` (quantas linhas valem), mas o código lê só `output.slice(0, 3)` e percorre as 100 linhas. Funciona porque as linhas vazias têm score 0 e caem no primeiro filtro. Mesmo assim, ler a 4ª saída evita trabalho e torna a intenção explícita.

SEÇÃO 07

Da previsão ao tiro

Na thread principal, cada mensagem `prediction` vira um clique simulado. A IA não tem um "atalho": ela usa o **mesmo** `handleClick` que o mouse do jogador usa, com as mesmas regras de balas e pontuação.

machine-learning/main.js

```
worker.onmessage = ({ data }) => {
    if (data.type === 'prediction') {
        container.updateHUD(data);                  // mostra score e (x, y) no HUD
        game.stage.aim.visible = true;
        game.stage.aim.setPosition(data.x, data.y); // mira nas coordenadas do stage
        const position = game.stage.aim.getGlobalPosition(); // stage → tela
        game.handleClick({ global: position });     // mesmo caminho do clique do mouse
    }
};
```

7.1 O que acontece dentro do `handleClick`

1. Se o ponto cair em *pause*, *mute*, *fullscreen* ou no link do criador de níveis, executa essa ação e para.
2. Se há balas, a onda não acabou e o jogo não está pausado: toca o tiro, `bullets -= 1` e chama `stage.shotsFired(ponto, level.radius)`.
3. `shotsFired` converte o ponto da tela de volta para coordenadas do stage (`getScaledClickLocation`) e mata todo pato vivo a menos de `radius` pixels (60 em todos os níveis). Cada pato vale `pointsPerDuck` = 100.

SISTEMAS DE COORDENADAS

Há **três sistemas de coordenadas** em jogo: o canvas extraído (onde o YOLO mede), o stage lógico 800 × 600 (onde os patos vivem) e a tela (o stage é esticado para o tamanho da janela por `scaleToWindow`). `setPosition` → `getGlobalPosition` → `getScaledClickLocation` faz a ida e a volta. Se o canvas extraído não estiver alinhado com o stage, todos os tiros saem deslocados pelo mesmo erro. Um `console.log(canvas.width, canvas.height)` no loop de captura é a primeira coisa a checar.

NA PRÁTICA

O raio de 60 px é generoso: um pato tem cerca de 70 × 60 px. Isso torna o sistema **tolerante a erro de localização**: o centro da caixa não precisa ser perfeito. O que limita a IA é encontrar o pato (recall) e chegar a tempo (latência), não a precisão do ponto.

SEÇÃO 08

Experimento: o que o modelo realmente vê

Para medir em vez de supor, o mesmo modelo e o mesmo pré-processamento do worker foram rodados em 40 cenas montadas com os sprites do jogo (árvore, grama e dois patos de cor, pose e posição aleatórias = 80 patos). Cada cena foi gerada em duas versões: com o céu azul que o jogador vê e com o céu transparente que `extract.canvas` entrega.

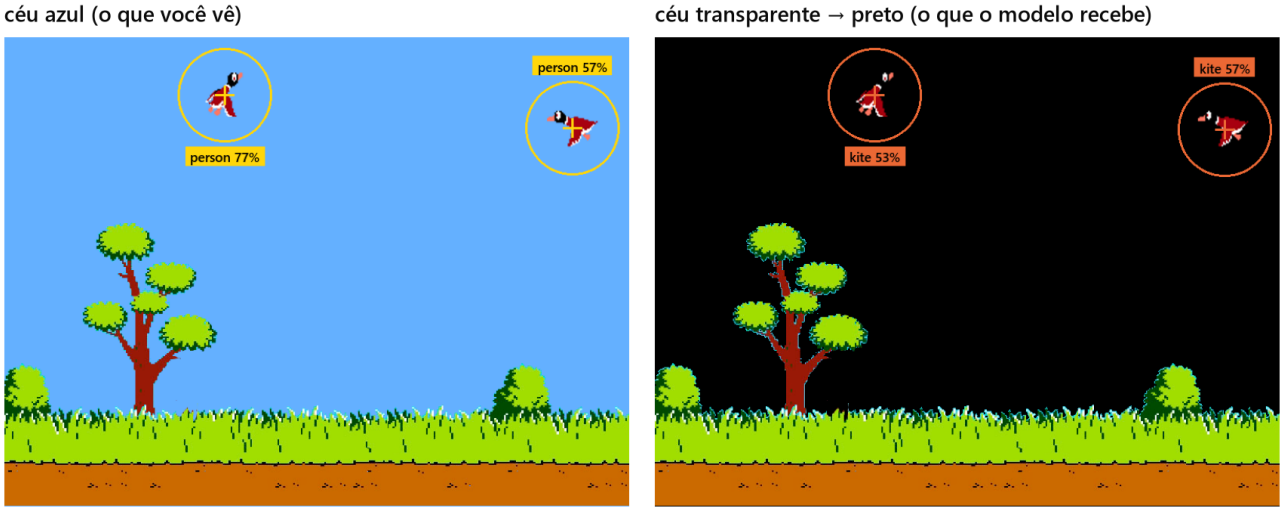


Figura 4. A mesma cena nas duas versões. Círculo = raio do tiro (60 px) em volta do centro previsto. No céu azul o modelo acha os dois patos, mas chama de **person**, que o filtro descarta. No fundo preto, os mesmos patos viram **kite** e seriam abatidos.

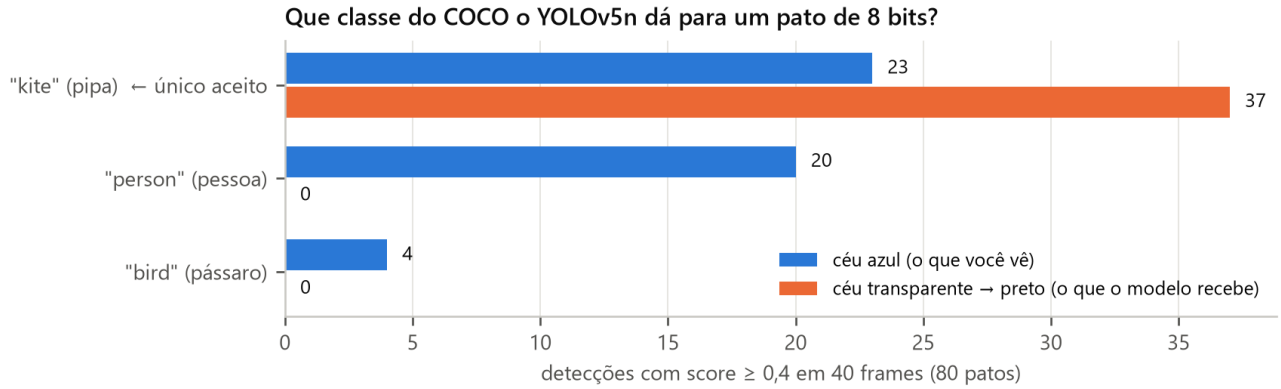


Figura 5. Classes atribuídas aos patos com confiança ≥ 0,4. No fundo preto o modelo é consistente: tudo vira kite. No azul, quase metade vira person ou bird.

8.1 Resultados com a configuração do projeto (kite, ≥ 0,4)

Métrica	Céu azul	Céu transparente → preto	Leitura
Precisão (tiros que acertaram um pato)	100%	100%	Nenhuma detecção caiu longe de um pato: zero tiros no vazio, na árvore ou na grama.
Recall (patos encontrados)	28,7% (23/80)	46,2% (37/80)	Mais da metade dos patos não é vista num frame isolado.

Métrica	Céu azul	Céu transparente → preto	Leitura
Recall aceitando qualquer classe	58,8%	46,2%	No azul, o filtro de classe joga fora metade dos acertos.
Recall do pato vermelho × preto	50% × 11,4%	69,4% × 27,3%	O pato preto tem pouco contraste, principalmente no fundo preto.

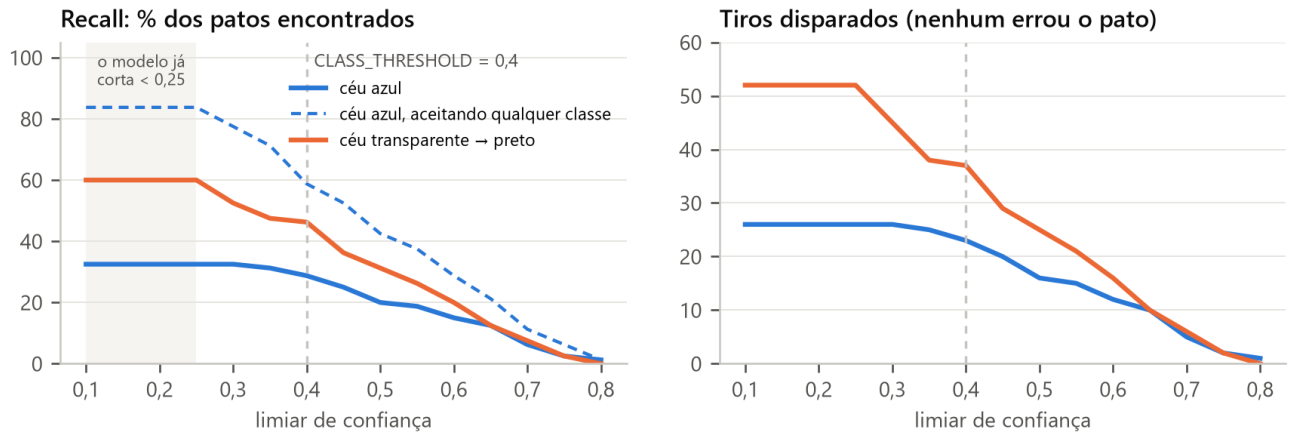


Figura 6. Efeito do limiar. Baixar de 0,4 para 0,25 aumenta o recall (46% → 60% no fundo preto) sem criar um único tiro errado neste teste. Abaixo de 0,25 nada muda: o NMS dentro do modelo já descartou essas caixas.

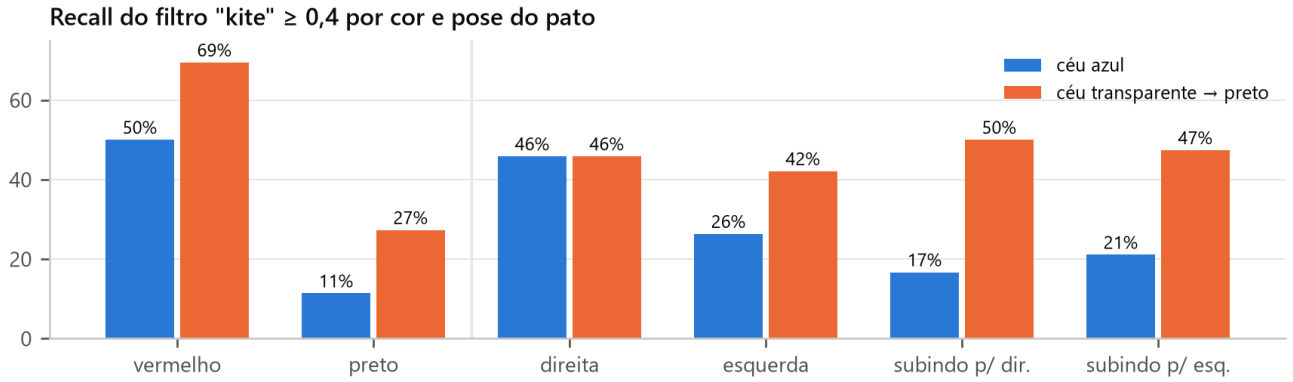


Figura 7. Recall por cor e pose. Patos vermelhos e voando na horizontal são os mais fáceis; patos pretos subindo são os mais difíceis.

A LIÇÃO DO EXPERIMENTO

O mesmo pato muda de classe quando muda o **fundo**. Isso é **domain shift** na prática: o modelo aprendeu correlações das fotos do COCO (pipas aparecem contra o céu; pessoas aparecem em todo tipo de cenário) e não o conceito de "pato". Qualquer mudança no pré-processamento (fundo, escala, cor) pode quebrar a regra `label == 'kite'` sem nenhum erro aparecer no console.

POR QUE A IA GANHA MESMO ASSIM

Recall de 46% por frame parece pouco, mas o jogo captura 5 frames por segundo e um pato fica vários segundos na tela. Se cada frame fosse uma chance independente, a chance de não ver o pato em 10 frames seguidos seria $0{,}54^{10} \approx 0{,}2\%$. Na prática os frames são parecidos entre si (não são independentes), então o ganho real é menor, mas é por isso que a IA ainda vence. O gargalo passa a ser outro recurso: **balas** (3 por onda no nível 1).

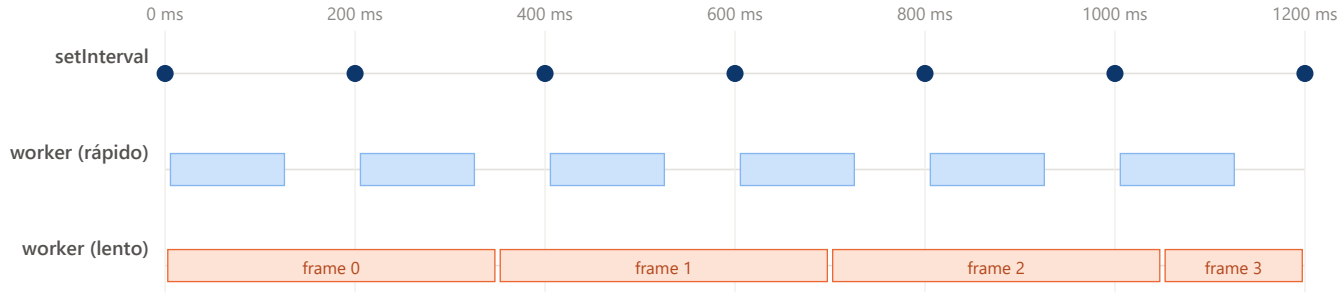
Como o experimento foi feito: os frames foram montados em Python com `dist/sprites.png` e a inferência rodou em Node.js com TensorFlow.js 4.22.0 (backend CPU), lendo o mesmo `model.json`. Um acerto é uma detecção cujo centro ficou a menos de 60 px do centro do pato, a mesma regra do `shotsFired`.

SEÇÃO 09

Avaliação crítica: o que um engenheiro de IA perguntaria

O projeto funciona e é um ótimo exemplo didático. Estas são as perguntas que separam “funciona na demo” de “funciona sempre”.

9.1 Latência e backpressure



Inferência > 200 ms: os frames se acumulam na fila do worker e a mira passa a mirar onde o pato ESTAVA.

Figura 8. Com inferência mais rápida que 200 ms (azul), cada frame é processado antes do próximo. Mais lenta (laranja), a fila cresce sem limite.

No backend WebGL de uma GPU comum a inferência do YOLOv5n costuma ficar abaixo de 200 ms, No backend CPU puro (sem GPU) ela levou **5,1 s** por frame neste teste: 25 vezes o intervalo, A solução é **backpressure**: só capturar um novo frame quando o worker responder.

Sugestão de melhoria

```
let busy = false
setInterval(async () => {
    if (busy) return                        // worker ocupado: pula este frame
    busy = true
    const bitmap = await createImageBitmap(game.app.renderer.extract.canvas(game.stage))
    worker.postMessage({ type: 'predict', image: bitmap }, [bitmap])
}, 200)
// no worker, após processar: postMessage({ type: 'done' })  →  na página: busy = false
```

9.2 Outras perguntas

Pergunta	Situação no código	Melhoria possível
O alvo ainda está lá quando o tiro sai?	Mira onde o pato estava no momento da captura.	Estimar a velocidade entre dois frames e mirar à frente (*lead*).
Quantas balas a IA gasta?	Uma mensagem = um tiro; duplicatas do mesmo pato gastam bala.	Um tiro por frame, no pato de maior score; ignorar alvos já abatidos.
E se o fundo mudar?	Céu rosa ao perder (`PINK_SKY_COLOR`) só afeta a tela; o modelo vê preto.	Testar em todas as cenas; não depender de uma classe “por acaso”.
A regra da classe é robusta?	`label != 'kite'` descarta patos vistos como person/bird.	Aceitar várias classes ou fazer *fine-tuning* com patos rotulados.
A versão do TF.js é fixa?	`@tensorflow/tfjs@latest` via CDN.	Fixar a versão (ex.: 4.22.0): uma atualização pode mudar o comportamento.

Pergunta	Situação no código	Melhoria possível
E a memória?	Entrada e saídas com `dispose`; pré-processo com `tidy`.	Correto. Monitorar `tf.memory().numTensors` para provar.
Como medir se melhorou?	Não há métrica no projeto, só o placar.	Logar acertos/tiros por onda e comparar versões com os mesmos níveis.

PRECISÃO × RECALL DEPENDE DO CUSTO DO ERRO
Neste jogo **precisão vale mais que recall**: um tiro errado custa uma bala de três, enquanto um pato não visto agora provavelmente será visto no próximo frame. É por isso que um limiar mais alto e uma única classe fazem sentido, mesmo perdendo detecções. Ajustar o limiar é escolher onde ficar nessa troca, e a escolha depende do **custo de cada tipo de erro**, não de uma regra universal.

SEÇÃO 10

Engenharia: IA rodando no navegador

Toda a inferência acontece na máquina do jogador. Nenhum servidor, nenhuma GPU na nuvem, nenhum frame enviado pela rede.

Peça	Papel	No projeto
TensorFlow.js	Executa o grafo do modelo com aceleração por WebGL (ou WebGPU / WASM / CPU).	`importScripts('…/tfjs@latest')` no worker
Graph model	Formato gerado pelo `tensorflowjs_converter` a partir de um SavedModel; pesos fatiados em shards de 4 MB para baixar em paralelo e cachear.	`tf.loadGraphModel(MODEL_PATH)`
Web Worker	Thread separada: a inferência não trava o `requestAnimationFrame` do PixiJS.	`new Worker(new URL(...))`
ImageBitmap	Imagem decodificada, pronta para a GPU e transferível entre threads.	`createImageBitmap(canvas)`
webpack	Empacota o jogo e o worker e copia a pasta do modelo para `dist/`.	`CopyWebpackPlugin`
PixiJS	Motor 2D do jogo; fornece `renderer.extract` para ler a cena como imagem.	`game.app.renderer`

10.1 Vantagens e custos de rodar no cliente

- **Privacidade e custo:** nenhum dado sai do navegador e o servidor não paga GPU. O mesmo argumento do exemplo 01.
- **Latência sem rede:** não há ida e volta a um servidor; o limite é a GPU do usuário.
- **Download:** 7,5 MB de pesos na primeira visita. Por isso o “n” (nano): o YOLOv5s teria cerca de 4 vezes mais parâmetros.
- **Heterogeneidade:** a mesma página roda rápido num PC com GPU e devagar num notebook sem aceleração. Backpressure (seção 9) é obrigatório em produção.

ATENÇÃO

O modelo e os rótulos são carregados com caminhos **relativos** (`yolov5n_web_model/model.json`). Eles funcionam porque o worker é servido da raiz de `dist/` e o `CopyWebpackPlugin` coloca a pasta lá. Abrir o `index.html` por `file://` não funciona: o `fetch` é bloqueado por CORS, como o README do projeto avisa.

SEÇÃO 11

Exercícios práticos

Cada exercício leva de 10 a 30 minutos. Rode `npm start` e acompanhe o console e o HUD da IA no canto superior direito.

1. Ver o que o modelo vê

No `onmessage` do worker, poste também `label` e `score` de **todas** as detecções ≥ 0,25 e mostre no HUD. Confira quantas vezes aparecem person e bird.

2. Mexer no limiar

Teste `CLASS_THRESHOLD` = 0,25, 0,4 e 0,6 no nível 1. Anote patos abatidos e balas gastas por onda. Relacione com a Figura 6.

3. Aceitar mais classes

Troque o filtro por `['kite', 'bird'].includes(label)`. O recall sobe? Aparecem tiros errados?

4. Fundo azul de verdade

Passe `{ target: game.stage, clearColor: '#64b0ff' }` para o `extract.canvas` (opção do PixiJS v8) e repita o exercício 1.

5. Backpressure

Implemente a flag `busy` da seção 9.1. Use o DevTools para simular CPU 6× mais lenta e compare com e sem.

6. Um tiro por frame

Mude o worker para mandar só a detecção de maior score de cada frame. Compare a taxa de acerto por bala.

7. Mirar à frente

Guarde a última posição prevista. Com duas posições e o tempo entre elas, estime a velocidade e some um deslocamento à mira.

8. Medir a memória

Mostre `tf.memory().numTensors` a cada 50 inferências. Remova o `tf.tidy` e veja o número crescer.

SEÇÃO 12

Glossário

Termo	Significado
Visão computacional	Área da IA que extrai informação de imagens e vídeos.
Detecção de objetos	Encontrar objetos numa imagem e devolver, para cada um, caixa, classe e confiança.
Bounding box	Retângulo que delimita um objeto; aqui em (x1, y1, x2, y2) normalizados.
YOLO	*You Only Look Once*: detector de um estágio que prevê todas as caixas numa única passada.
Backbone / neck / head	Partes de um detector: extração de características, fusão de escalas e previsão das caixas.
Anchor	Formato de caixa pré-definido a partir do qual a rede prevê ajustes.
NMS	Non-Maximum Suppression: remove caixas duplicadas que se sobrepõem, mantendo a de maior confiança.
IoU	Intersection over Union: área da interseção dividida pela área da união de duas caixas; usada pelo NMS.
COCO	Dataset com 80 classes de objetos do dia a dia, usado para treinar o YOLOv5.
Limiar (threshold)	Confiança mínima para aceitar uma detecção. Mais alto: mais precisão, menos recall.
Precisão	Das detecções aceitas, quantas estavam certas.
Recall	Dos objetos que existiam, quantos foram encontrados.
Zero-shot / transfer	Usar um modelo numa tarefa ou domínio para o qual ele não foi treinado.
Domain shift	Diferença entre os dados de treino (fotos) e os de uso (pixel art), que degrada o modelo.
Fine-tuning	Continuar o treino de um modelo pronto com dados do novo domínio.
Tensor	Array multidimensional de números com formato (shape), ex.: [1, 640, 640, 3].
NHWC	Ordem das dimensões de imagem no TF: lote, altura, largura, canais.
Letterbox	Redimensionar mantendo a proporção e completar com bordas.
Graph model	Modelo TF.js congelado, só para inferência, convertido de um SavedModel.
Warmup	Inferência falsa no início para compilar shaders e alocar memória antes do uso real.
Web Worker	Thread separada do navegador; conversa com a página por `postMessage`.
Objeto transferível	Objeto (ImageBitmap, ArrayBuffer) movido entre threads sem cópia.
Backpressure	Mecanismo para o produtor não enviar trabalho mais rápido do que o consumidor processa.
Generator	Função `function*` que entrega valores sob demanda com `yield`.

SEÇÃO 13

Perguntas de revisão

Responda por escrito, sem consultar o guia. Depois confira o gabarito na página seguinte.

1. Por que o projeto usa detecção de objetos e não classificação de imagens?
2. O que significa dizer que o projeto usa o YOLOv5n de forma zero-shot? Que risco isso traz?
3. Descreva o formato do tensor de entrada do modelo e a função de cada operação de `preprocessImage`.
4. Por que `tf.tidy` é necessário no TF.js e o coletor de lixo do JavaScript não basta?
5. Por que o projeto filtra a classe “kite” e não “bird”?
6. Explique o que é NMS e por que, neste projeto, ele não aparece no código JavaScript.
7. Por que o modelo é executado com `executeAsync`? Para que serve o warmup?
8. Como uma caixa normalizada (x1, y1, x2, y2) vira o ponto do clique? Faça a conta para x1 = 0,40, x2 = 0,48 e largura 800.
9. Para que serve o Web Worker e o que significa o segundo argumento de `postMessage(msg, [bitmap])`?
10. Por que o céu chega preto ao modelo, e qual efeito isso teve nas classes detectadas?
11. No experimento, a precisão foi 100% e o recall 46%. Explique cada número com suas palavras.
12. Neste jogo, você prefere aumentar a precisão ou o recall? Justifique pelo custo de cada erro.
13. O que acontece se a inferência demorar 500 ms com `setInterval` de 200 ms? Como resolver?
14. Por que redimensionar 800 × 600 para 640 × 640 deforma a imagem, e por que as coordenadas finais continuam certas?
15. Cite duas formas de fazer a IA perder menos patos sem trocar de modelo, e uma trocando de abordagem.

Gabarito

1. Porque para atirar é preciso saber **onde** está o pato. Classificação diz só o que há na imagem; detecção devolve caixas com posição.
2. O modelo é usado sem nenhum treino no domínio do jogo: foi treinado com fotos reais (COCO) e aplicado em pixel art. O risco é o domain shift: ele pode errar a classe ou não ver o objeto, e qualquer mudança visual pode quebrar a regra escolhida.
3. [1, 640, 640, 3] em float 0..1. `fromPixels` cria o tensor RGB; `resizeBilinear` ajusta ao tamanho fixo do modelo; `div(255)` normaliza como no treino; `expandDims(0)` adiciona a dimensão do lote.
4. Os tensores vivem na memória da GPU (texturas WebGL), fora do alcance do coletor de lixo. Sem `tidy`/`dispose` cada frame vazaria vários MB até o navegador travar.
5. Porque foi a classe que o modelo mais atribuiu aos patos na prática. O COCO não tem pato e, para um pato pixelado contra fundo liso, a classe “pipa” disparou mais e com mais consistência do que “pássaro”.
6. Várias células da grade detectam o mesmo objeto; o NMS mantém a caixa de maior confiança e remove as que têm IoU alto com ela. Aqui ele já foi exportado dentro do grafo do modelo, que entrega no máximo 100 caixas filtradas.
7. Porque o grafo tem operações de fluxo de controle (os laços do NMS), que só a execução assíncrona suporta. O warmup compila os shaders e aloca memória antes do jogo, evitando um engasgo no primeiro frame real.
8. x1 = 0,40 × 800 = 320; x2 = 0,48 × 800 = 384; largura = 64; centro = 320 + 32 = **352**. O mesmo vale para y com a altura.
9. O worker roda a inferência numa thread separada, sem travar a animação do jogo. O array `[bitmap]` lista objetos **transferidos**: passam para o worker sem cópia e deixam de ser usáveis na página.
10. `extract.canvas(game.stage)` não inclui a cor de fundo do renderer, então o céu fica transparente; `fromPixels` descarta o alfa e o transparente vira preto. No teste, com fundo preto todos os patos detectados viraram kite; com fundo azul muitos viraram person.
11. Precisão 100%: todas as detecções aceitas estavam em cima de um pato (nenhum tiro no vazio). Recall 46%: de cada 100 patos, 46 foram encontrados num frame isolado.
12. Precisão: um tiro errado gasta uma bala (3 por onda no nível 1), enquanto um pato não visto agora tende a ser visto num dos próximos frames.
13. Os frames se acumulam na fila do worker, a resposta chega cada vez mais atrasada e a mira aponta para onde o pato estava. Solução: backpressure (só capturar quando o worker estiver livre) ou trocar `setInterval` por um laço que espera a resposta.
14. Porque 4:3 vira 1:1: a largura encolhe mais (800 → 640) do que a altura (600 → 640 cresce). As coordenadas saem normalizadas e são multiplicadas pela largura e altura originais, o que desfaz a deformação.
15. Sem trocar de modelo: baixar o limiar para ≈ 0,25; aceitar também bird/person; alimentar o modelo com o fundo azul; mirar à frente do pato. Trocando de abordagem: fine-tuning do YOLO com frames rotulados do jogo, ou ler o estado do jogo diretamente.

SEÇÃO 14

Resumo para revisar em 5 minutos

Tema	Ideia que você precisa lembrar
Problema	Agente que joga pela imagem: perceber (detectar) → decidir (filtrar) → agir (clicar).
Abordagem	Detecção de objetos com YOLOv5n pré-treinado no COCO, sem treino: transferência zero-shot.
Pipeline	extract.canvas → ImageBitmap → worker → tensor → executeAsync → filtro → postMessage → mira → handleClick.
Entrada	[1, 640, 640, 3], float 0..1, NHWC. fromPixels + resizeBilinear + div(255) + expandDims, tudo em tf.tidy.
Modelo	Graph model, 1,88 M parâmetros, NMS embutido, 4 saídas (boxes, scores, classes, valid). executeAsync + warmup.
Filtro	score ≥ 0,4 e classe "kite"; caixa normalizada × largura/altura → centro; um yield por pato.
Ação	Mesmo handleClick do mouse; raio de 60 px; 100 pontos por pato; balas limitadas.
Experimento	Precisão 100%; recall 46% (fundo preto) e 29% (azul). No azul o pato vira person.
Riscos	Domain shift, regra de classe frágil, fila sem backpressure, mira atrasada, tfjs@latest.
Engenharia	Tudo no navegador: Web Worker, objeto transferível, dispose/tidy, modelo copiado pelo webpack.

A LIÇÃO MAIS IMPORTANTE DESTE PROJETO

Um modelo pronto raramente é a solução inteira. O valor de engenharia deste projeto está em **tudo ao redor do modelo**: capturar e preparar a entrada certa, entender o que a saída significa nesse domínio, transformar a previsão numa ação e **medir** o resultado. Trocar o fundo da imagem mudou a classe dos patos: só quem mede descobre isso.

Para ir além

- Documentação do TensorFlow.js e do conversor de modelos: tensorflow.org/js
- Repositório e documentação do YOLOv5 (Ultralytics): github.com/ultralytics/yolov5
- Artigo original do YOLO: Redmon et al., *You Only Look Once: Unified, Real-Time Object Detection* (2016).
- Dataset COCO e suas 80 classes: cocodataset.org